THE UNIVERSITY of ADELAIDE

Hybrid Recommender System for Grocery Retail: Leveraging Frequent Pattern Mining and Collaborative Filtering with Recency-Aware Personalization

Group 1
Phuong Quoc Anh To – a1915616
Nivedhitha Muruganandam – a1917750
Sonali Goswami – a1934448

April 27, 2025

Table of Contents

Contents

Executive Summary

The objective of this report is to outline the development and evaluation of a recommender system, which recommends personalized grocery items based on customers' purchase histories to boost sales. Our developed system comprises two main components: **frequent pattern mining** and **collaborative filtering**, in combination with **recency-aware personalization** to improve recommendation quality. In this work, we explore how **mining frequent patterns** can generate tailored recommendations based on user preferences and behaviour. The report concludes with suggestions for further tuning, including potential improvements in scalability and personalization that meet real-world criteria.

1 Introduction

Recommender systems have long been recognized as a widespread method across the world, with prominent applications in major commercial platforms such as Amazon, Netflix, and Spotify [1]. These systems play a key role in enhancing customer experience through personalized recommendations that raise the level of user engagement and sales. According to Falk, a recommender system calculates and provides relevant content to the user based on knowledge of the user, content, and interactions between the user and the item [2].

2 Related Work

Over the past decades, researchers have studied and developed recommender systems in several ways. Specifically, combining frequent pattern mining with collaborative filtering has been extensively investigated in recommender systems to use both transactional patterns and user-item interaction data for enhanced recommendation accuracy [3]. Other research has introduced hybrid models that jointly utilize pattern mining and collaborative filtering to address challenges such as data sparsity and cold-start problems, achieving better performance than either method alone [4].

In this paper, we not only utilize the common methods of pattern mining and collaborative filtering but also seek to fill the research gap in this area by introducing the Recency-Aware Personalization approach.

3 Exploratory Analysis

3.1 Dataset description

Our research utilized a grocery retail transactional dataset that has been specially structured for the purpose of developing and assessing recommendation systems. The entire dataset comprises approximately 38,600 records, which is arranged chronologically by transaction date. The dataset is split into two identical segments: the training set, which comprises the initial portion of approximately 19,300 records, and the test set, which includes the latter portion containing roughly 19,300 records.

Each transaction record includes several attributes that are largely self-explanatory, such as:

- **User_id**: A unique identifier for each customer.
- **itemDescription**: The description of the purchased item.
- **Date**: The purchase date of the transaction.
- **year, month, day**: Decomposed date components consistent with the `Date` column.
- **day_of_week**: An integer encoding where Monday = 1 and Sunday = 7.

3.2 User Behavior Analysis

The exploratory data analysis reveals important patterns in user behavior and item popularity. Fig. 1 shows that the daily transaction numbers are relatively stable, with most of the days having a total of 15 to 35 purchases.

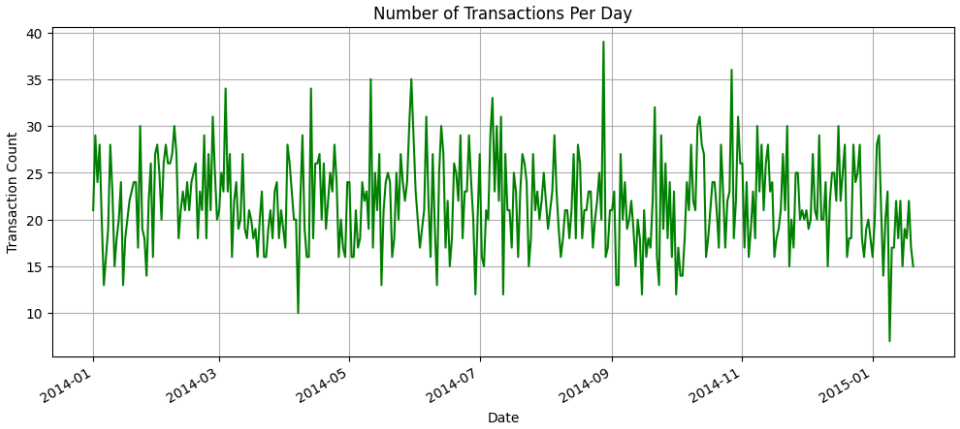
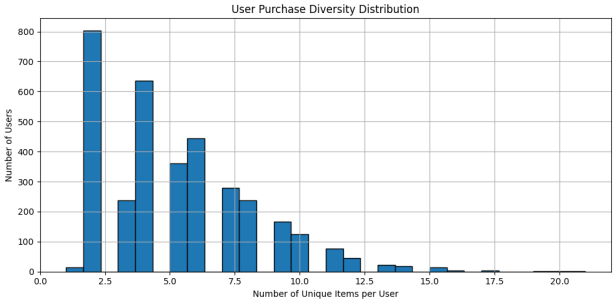
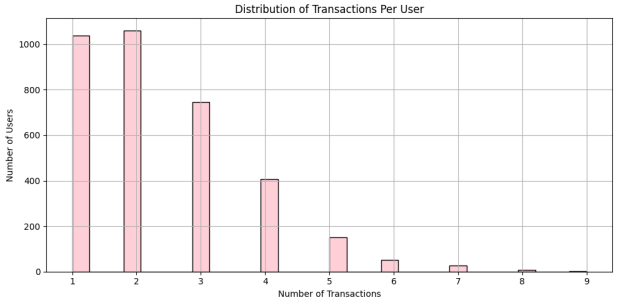


Figure 1: Daily purchase

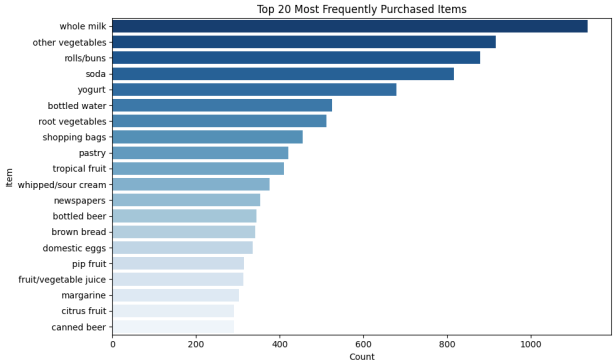
However, Fig. 2b indicates that the majority of customers have made only one or two transactions, and the purchasing diversity chart in Fig. 2a shows that over 800 users bought only two unique items, which indicates that the majority of users are only interacting with a limited number of items. This sparsity might pose a significant challenge for collaborative filtering, as collaborative filtering relies on shared behavior across users [5].



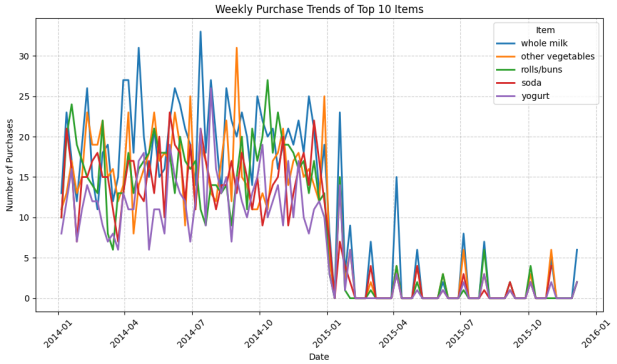
(a) Unique items per user



(b) Transactions per user



(c) Top 20 purchased items



(d) Weekly trend of top items

Figure 2: User behavior and item popularity analysis

When looking at popular items, items like whole milk, other vegetables, rolls/buns, soda, and yogurt are the most popular items. Fig. 2c and these products have consistent weekly demand, which is followed by a sudden decline post 2015. Insights from the analysis indicate that the users are repeat purchasing certain items, there is minimal overlap between user purchases, while certain products exhibit consistent demand.

4 Implementation

4.1 Task 1: Frequent pattern mining

4.1.1 Methodology for selecting appropriate metric and algorithm

In this section, we justify our choice of the FP-Growth algorithm as the optimal model to mine frequent itemsets along with support metric.

Input: Training set containing transactional data.

Output: A list of top 5 frequent itemsets, which is ranked by recency-weighted scores.

Threshold Selection Reasoning: To establish the best support threshold for FP-Growth, we evaluated the number of patterns generated at different thresholds (Fig. 3). A threshold of 0.001 generated over 500 patterns, while also adding noise to the analysis by generating many infrequent itemsets. Other thresholds below 0.005 resulted in too few patterns (less than 50) and therefore missing meaningful co-purchase behaviors that exist in grocery retail. Our selected threshold of 0.005, resulting in 100 patterns approximately, justifies the threshold by balancing the number of missed patterns whilst reducing noise. It is also important to ensure computational efficiency for the case study, as well as ensuring trustworthy recommendations to grocery customers when benchmarking parameters, as Han et al. (2011) suggest not to go too low (as it adds noise) or too high (as it will lose some key patterns) [6] and Aggarwal (2016) recommends balancing threshold risk in terms of retail scenario [7].

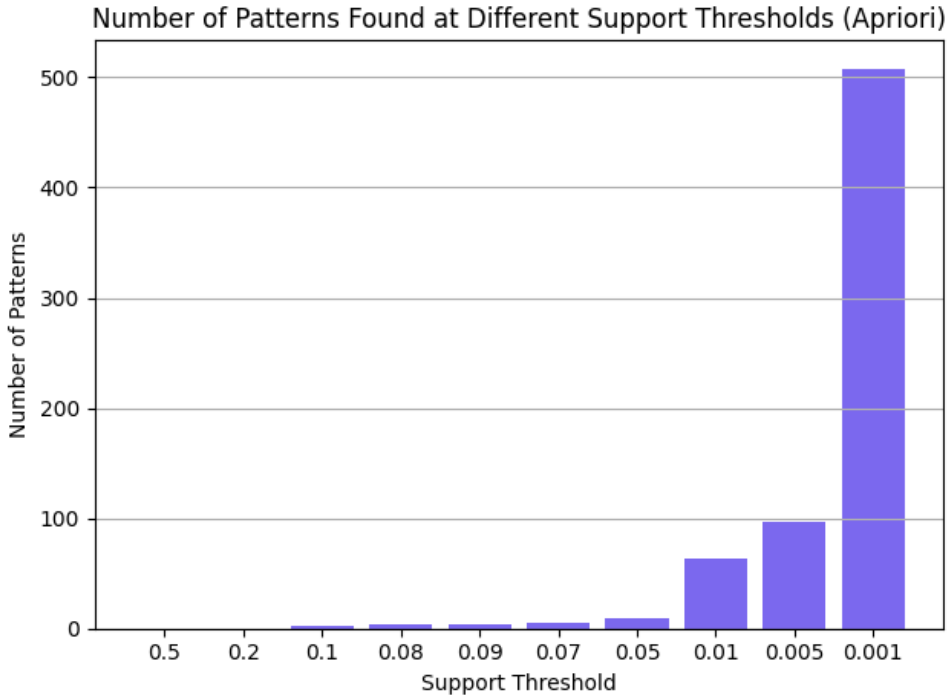


Figure 3: Threshold experiments

Experimentations and Evaluation: We conducted several experiments comparing Apriori and FP-Growth with different configurations. This includes setting with and without recency weighting, with the chosen threshold 0.005 that has already been justified above. The metrics we used in this work are runtime and memory usage, which represent computational efficiency, as depicted in Table 1.

- **Observations:** We chose FP-Growth with Recency as our final model because it gave us a good balance between performance and efficiency. According to the table, it had the lowest memory usage (16.88 MB) and almost the same runtime (1.88s) as regular FP-Growth. What made it unique was that we were able to use recency scores to increase the relevance of our recommendations by reflecting on the age of each item purchased whilst also maintaining speed and memory. Not only was it more memory efficient than Apriori models, but it was also significantly quicker, while producing results with comparable high precision. Thus, it was the most efficient practical and useful model for our analysis.
- **Scalability and Efficiency Advantage:** While FP-Growth's runtime is high compared to previous models, it is expected to perform better in terms of memory efficiency. Memory efficiency is important for our grocery retail recommender, because of the nature of the potentially large transactional datasets and ability to scale down the memory overhead; especially as grocery retail has dense datasets with frequent repeated purchases, such as whole milk as shown in Fig. 2c) [8]. To address the high runtime issue, we provided frequent patterns and the collaborative filtering user-item matrix in advance using an eager approach to all the user interactions, thus minimizing the impact of runtime and providing real-time recommendation without any change in user interaction.
- **Recency-Aware Personalization Benefit:** Finally, FP-Growth + Recency aligned with our data mining purpose of recency-aware personalization, as recency can prioritise recent user behavioural data with shared value (increasing similarity) to promote relevancy in a recommendation context, an idea supported by Koren et al. (2009) who concluded that the recent interactions are a better representation of the user's current preferences [9]. It is apparent, in terms of memory efficiency, scalability of recency, FP-Growth + Recency was ultimately determined to be the best model for frequent pattern mining in our hybrid recommender system.

Table 1: Computational Performance of FP-growth vs. Apriori with and without Recency

Algorithm	Support Threshold	Runtime (s)	Memory Usage (MB)
Apriori	0.005	0.08	66.20
FP-Growth	0.005	1.90	16.89
Apriori + Recency	0.005	0.07	66.20
FP-Growth + Recency	0.005	1.88	16.88

4.1.2 FP-Growth Algorithm

The FP-Growth algorithm provides an alternative way to mine a frequent itemset by compressing transaction records using a special graph data structure called FP-Tree. FP-Tree can be thought of as a transformation of the dataset into graph format. Rather than the generate and test approach used in the Apriori algorithm, FP-Growth first generates the FP-Tree and uses this compressed tree to generate the frequent itemsets. The efficiency of the FP-Growth algorithm depends on how much compression can be achieved in generating the FP-Tree [6]. Mathematically, the conditional pattern

base (**CPB**, i.e., *Conditional Pattern Base*) for an item **i** is derived from the FP-tree to recursively generate conditional FP-trees:

$$\text{CPB}(i) = \{\text{prefix path of } i \mid \text{frequency}(i)\} \tag{1}$$

FP-Growth significantly reduces computational overhead and is highly effective for dense datasets and large-scale transactional data, making it suitable for practical recommender system applications [8]. FP-Growth employs a **depth-first search** algorithm to explore frequent itemsets recursively and efficiently [6].

Algorithm 1 FP-Growth Algorithm

Require: Dataset D, minimum support threshold σ
Ensure: All frequent itemsets
Construct the FP-tree from D
for all items i in FP-tree (in support ascending order) **do**
    Construct conditional pattern base for i
    Construct conditional FP-tree from conditional pattern base
    Recursively mine the conditional FP-tree
end for
return All frequent itemsets

4.2 Task 2: Collaborative filtering

Collaborative filtering (CF) is among the most widely used methods in recommender systems since users with similar past behaviours or preferences are likely to show similar behaviours in the future [10]. Our hybrid recommender system's collaborative filtering component uses cosine similarity metrics and the K-Nearest Neighbours (KNN) item-based method. This approach finds relationships between grocery products by means of collective user purchase patterns, allowing customized recommendations outside of simple associations.

Algorithm 2 Item-based Collaborative Filtering using KNN

Require: Training dataset D, user ID u, number of neighbors k
Ensure: Top-N recommended items
Build user-item matrix from D
Train KNN model on user-item matrix with cosine similarity
if u not in user-item matrix **then**
    return Empty recommendation list
end if
Retrieve items previously interacted by user u
for all item i in user's interacted items **do**
    Find k nearest neighbors for item i
    for all neighbor item j not interacted by user **do**
        Accumulate similarity score for item j
    end for
end for
Rank candidate items by cumulative similarity scores
return Top-N highest-ranked items

4.2.1 Collaborative Filtering Training

Sparsity problem User-based collaborative filtering requires a user-item matrix. However, the user-item matrix was highly sparse, with 98.86% of the matrix having 0 elements, i.e., 3.33% of users have overlapping purchase history. Such sparsity complicates identifying user neighbors [2]. To address this item, based collaborative filtering was used, which is robust in sparse settings [11].
The train set was split into training and development sets with an 80-20 split. The collaborative filtering was developed by constructing a user-item matrix where each row represents an item and the column represents the user on the training set. The matrix entries have non-zero values that indicate the number of times an item was purchased by a user. A zero value means that the item was not purchased by the user. A K-nearest neighbors model was employed using cosine similarity as the distance metric and a brute-force search algorithm. This configuration was selected post experimenting with different metrics such as Jaccard similarity with binary user-item matrix and user-item matrix with frequency of purchase and recency factored (appendix A). The best configuration was then evaluated using the dev set and then chosen based on precision@k and recall@k (k = 5) scores. Here precision and recall are chosen to understand how accurate and complete the recommendations are, helping evaluate both the quality and coverage of the system [12].

The best-performing model was the frequency-aware item-user matrix with cosine similarity, achieving a precision of 0.0938 and a recall of 0.2082.

4.2.2 Collaborative Filtering model evaluation

Two main recommendation strategies were evaluated. A standard user-item-based collaborative filtering model and a hybrid model that utilizes frequent patterns from frequent itemset mining. The CF model relies only on item-based similarity, which was derived from historical interactions, to make recommendations. In contrast, the hybrid strategy combines the CF-based recommendation with results from mining frequent patterns, which identifies the common co-purchased items that appear in the dataset. The model filters out items already seen by the user and prioritizes those that appear in both the user's neighborhood and the globally frequent itemsets and de-prioritizes items with lower confidence.

Model	**Precision@5**	**Recall@5**	**Hit Rate@5**	**NDCG@5**
Collaborative Filtering (CF)	0.0848	0.1558	0.3545	0.1387
Hybrid Model	**0.112**	**0.2045**	**0.4627**	**0.1679**

Table 2: Performance Comparison of CF and Hybrid Models (Top-5 Recommendations) on development set

The models were trained on the training set and evaluated on the development set. The model performance was compared using Precision@k, Recall@k, Hit Rate@k, and NDCG@k (k = 5).

Precision@k and Recall@k were chosen to evaluate the overall relevance of the recommendation and if the model retrieved all possible items, respectively. Hit Rate and NDCG were selected to check if the user received one relevant item and NDCG@k for the ranking quality, which rewards relevant items that appear early on the recommendation list [13].

The results showed that the hybrid model outperformed the standalone CF model, achieving higher scores across all metrics as presented in Table 2, which indicates improved recommendation relevance by leveraging personalized recommendation and global item patterns.

The hybrid model performed better than the standard model due to the diverse purchase patterns among customers shown in Fig. 2a. Since most users tend to interact with a small number of unique

items, the collaborative filtering model has difficulty determining the neighbors. The hybrid model mitigates this by using globally frequent itemsets, which helps in enhancing the recommendations even when user histories are sparse or repetitive.

4.3 Task 3: System integration

4.3.1 Literature review

Recommendation systems have historically leveraged frequent pattern mining to represent co-occurrence relationships from transactional data. When discussing frequent itemsets, Aggarwal (2016) argues frequent itemsets can overcome sparsity by identifying global patterns to augment user-based recommendations [7]. Consider a market basket analysis for example, if the frequent itemset bread, butter was identified, the frequent pattern could suggest the user buy ”butter” with bread, which might be valuable to a user with limited purchase history; in this case, the corresponding recommendation would not be user-based, but still able to suggest relevant or complimentary items.

Lin et al. (2002) introduced a hybrid model that integrates FP-growth and collaborative filtering. They found that the model produced a relatively accurate recommendation system in retail contexts. They also found that frequent patterns were especially useful in real-world situations, especially if a user was in a cold-start problem [14].

Koren et al. (2009) advanced and expanded the discussion of time to recommendation systems, arguing that users' preferences change over time and that recent interactions are normally more indicative of the user's current preference. Their approaches utilizing time-awareness demonstrated that they could improve recommendation accuracy by modeling recency and temporal trends directly in collaborative filtering algorithms [9].

4.3.2 Practical implications for our work

- **Hybrid approach:** Inspired by the work of Lin et al., we decided to use frequent itemsets to boost collaborative filtering recommendations. This helps us to combat the sparsity issue in our dataset and provides fallback for new users.
- **Recency weighting:** We apply the concept of recency-aware personalization and temporal dynamics that was highlighted by the work of Koren et al. (2009), to weight recent user feedback higher than feedback that is less recent.

4.3.3 Integration code

The integration code merges collaborative filtering and frequent pattern mining to provide personalized recommendations. It processes the transactional dataset by first preparing the user-item interactions and mining the frequent itemsets, accounting for their recency so that we can provide recommendations based on recent user activity. A user interface has been designated for entering the User ID and selecting the recommendation method (using frequent patterns or only collaborative filtering). If the user does not have a history of activity in the dataset, we derive recommendations directly from the itemsets since collaborative filtering alone cannot handle new users. The implementation also ensures the computations are efficient and that the models are pre-built in eager approach. The system is then evaluated on test set to understand overall performance using similar metrics that were used to evaluate development set.

Fig. 4 outlines the architecture of the hybrid recommender system which loads the model in eager approach, confirming how historical transactions as data are processed and in turn, frequent itemsets

are created through FP-growth and a user-item matrix to create collaborative filtering, and then implemented into the recommendation system to process user input and ultimately provide recommendations. Fig. 5 contains the flow of the hybrid recommender system depicting the decision-making process the system takes, wherein it decides whether to use frequent itemsets, next creating frequent patterns, generating recommendations from collaborative filtering, and lastly, re-ranking the items from overlapping frequent patterns to increase relevance.

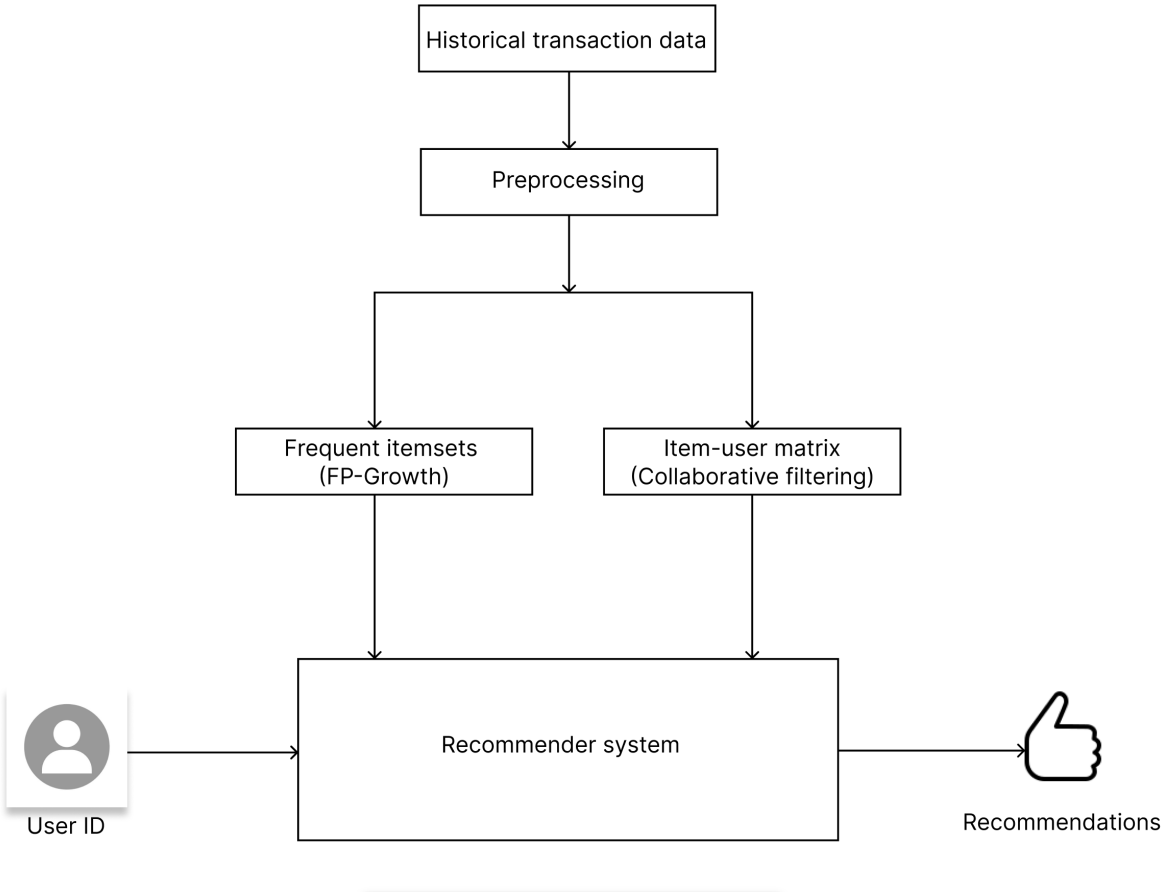


Figure 4: Architecture of the eager hybrid recommender system

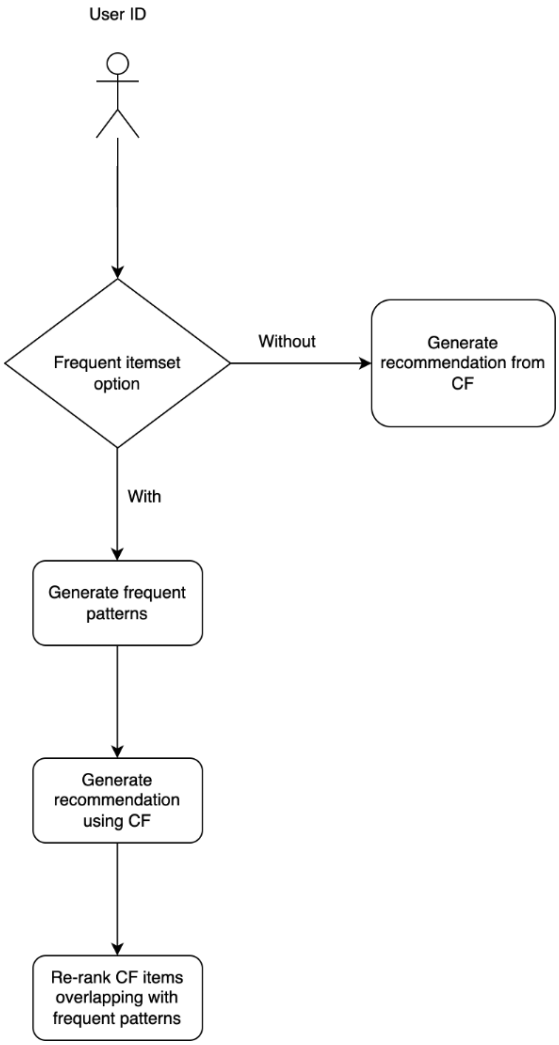


Figure 5: Flow diagram for hybrid recommender system

4.4 Experimental evaluation

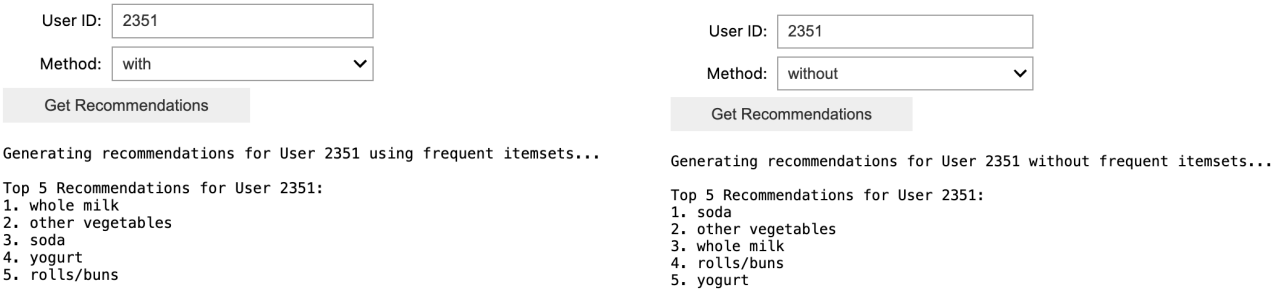


Figure 6: Comparison of recommendation outputs with and without frequent itemsets for User ID 2351

4.4.1 Program implementation

To demonstrate the system's functionality, we tested it using both methods with arbitrary User ID (2351) from the dataset. After running different test cases, we received outputs displaying in a simple User Interface created by Jupyter Widgets, as shown in Fig. 6. These examples show that incorporating frequent itemsets can adjust the recommendation order, prioritizing items that align with frequent patterns, while still being relevant to the user's transaction history.

4.4.2 Evaluation Results and Discussion

For the purpose of examining the performance of our hybrid recommender system, we evaluated the hybrid model (which utilizes frequent itemsets) and the stand-alone collaborative filtering (CF) model (that does not utilize frequent itemsets) against the test set. We assessed each model's performance with metrics of Precision@k, Recall@k, Hit Rate@k, NDCG@k (k = 5), runtime and peak memory, summarized in Table 3.

Table 3: Performance Comparison of Hybrid and CF Models on Test Set

Model	Precision@5	Recall@5	Hit Rate@5	NDCG@5	Runtime (s)	Peak Memory (MB)
CF (Without Frequent Itemsets)	0.2014	0.1886	0.6476	**0.2480**	**89.66**	26.48
Hybrid (With Frequent Itemsets)	**0.2040**	**0.1911**	**0.6529**	0.2363	102.05	**26.43**

It can be clearly seen from the Table 3 that the hybrid model outperforms the standalone CF model with respect to most metrics (refer to the bold figures in the table that represents better metrics). We observed slight improvements in the metrics for Precision@5, Recall@5, and Hit Rate@5, these improvements were 1.29%, 1.33%, and 0.82%, respectively. Although these enhancements were modest, they still highlight the potential of leveraging frequent itemsets in order to mitigate the sparsity issue commonly observed in CF systems [7]. However, it is noticeable that the stand-alone CF model had better performance than the hybrid model regarding the NDCG@5 metric (0.2480 vs. 0.2363). This indicates a better structure of recommendations based purely on CF similarity metrics [9]. The hybrid model was running slower than the CF model for the runtime measure (89.66s vs. 102.05s), which indicates the computational overhead introduced by frequent itemset mining. [6]. Consequently, the hybrid model required less memory (26.43 MB) to perform its recommendations, compared to the memory usage of the CF model (26.48 MB), this suggests that incorporating frequent itemsets makes the system more memory-efficient. This finding has already been emphasized by previous studies, especially the work of Zaki et al. (2014), which mentioned the memory benefits of compact pattern mining algorithms such as FP-Growth. [8].

5 Conclusion and future work

This paper describes the implementation of our hybrid recommender system that employs hybrid approach, which comprises two main components: frequent pattern mining and collaborative filtering, in combination with recency-aware personalization to improve recommendation quality, also highlighting both its strengths and limitations. However, we identified an issue that could significantly hinder the real-world recommendation performance, especially in resource-constrained environments: the computational overhead that our system faced. For future work, we plan to optimize runtime efficiency and refine recommendation ranking accuracy to make our recommender system adapt to real-world market needs.

References

[1] Ferdos Fessahaye, Luis Perez, Tiffany Zhan, Raymond Zhang, Calais Fossier, Robyn Markarian, Carter Chiu, Justin Zhan, Laxmi Gewali, and Paul Oh. T-recsys: A novel music recommendation system using deep learning. In *2019 IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–6, 2019.

[2] Kim Falk. *Practical Recommender Systems*. Manning Publications, 2019.

[3] Robin Burke et al. *Hybrid Web Recommender Systems*. Springer, 2027.

[4] Xinyu Liu et al. Hybrid recommendation system combining collaborative filtering with utility mining. *Scientific Reports*, 14(29390), 2024.

[5] Feng Zhang and Hui-You Chang. A collaborative filtering algorithm embedded bp network to ameliorate sparsity issue. In *2005 International Conference on Machine Learning and Cybernetics*, volume 3, pages 1839–1844 Vol. 3, 2005.

[6] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann Publishers, 3rd edition, 2011.

[7] Charu C. Aggarwal. *Recommender Systems: The Textbook*. Springer, 2016.

[8] Mohammed J. Zaki and Wagner Meira. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2014.

[9] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

[10] J. Ben Schafer, John A. Konstan, and John Riedl. Recommender systems in e-commerce. In *Proceedings of the 1st ACM Conference on Electronic Commerce (EC-99)*, pages 158–166. ACM, 1999.

[11] Ronakkumar Patel and Priyank Thakkar. Ibcfaicdr: Auxiliary data-driven item-based collaborative filtering in cross-domain rss to address user cold start problem. *Results in Engineering*, 24:103257, 2024.

[12] Edgar Sarmiento, Cesar A., Lehi Quincho, and Jair F. Multi-objective evolutionary programming for developing recommender systems based on collaborative filtering. *International Journal of Advanced Computer Science and Applications*, 11, 01 2020.

[13] Asela Gunawardana and Guy Shani. Evaluating recommender systems. In *Recommender Systems Handbook*, pages 265–308. Springer, 2015.

[14] Wei Lin, Silvia A. Alvarez, and Carolina Ruiz. Efficient adaptive-support association rule mining for recommender systems. *Data Mining and Knowledge Discovery*, 6(1):83–105, 2002.

[15] Wikipedia contributors. Separation of concerns, 2025.

[16] Martin Fowler. Modular programming: The secret to efficient code, 2025.

Appendix

A Collaborative Filtering Experiments

Model	Precision@5	Recall@5
Binary Purchase Matrix – Jaccard Similarity	0.0926	0.2053
Frequency-Aware Item user Matrix – Cosine Similarity	0.0938	0.2082
Recency and Frequency Weighted Matrix – Cosine Similarity	0.0621	0.1376
Item-Based CF with Recency-Based Weighting – Cosine Similarity	0.0337	0.0713
User based CF – Jaccard Similarity	0.0366	0.0654

Table 4: Performance Comparison of Recommendation Models (Top-5 Recommendations)

Contributions and Reflections

A.1 Task 1

I contributed approximately 33.3% to the final recommender system code and report. This includes my work on data preprocessing, implementing and benchmarking the frequent pattern mining models, and writing sections of the experimental analysis. I also helped integrate the final FP-Growth + Recency model into the system.

At the beginning of this project, I had a basic understanding of association rule mining, but I had never implemented it fully on real-world data. **In Version 1,** I focused on cleaning and preparing the dataset. I removed any rows that were missing a User_id or itemDescription, because incomplete records would not be useful for analysis. Then, I ensured that User_id was treated as an integer and used pandas to properly format the Date column. I also created a Transaction_ID by combining the User_id and Date so that each shopping session could be uniquely identified. I added a day_of_week column to help with later analysis.

In Version 2, I initially designed a custom metric called User Personalized Pattern Score. My idea was to manually score patterns by giving more importance to items that a specific user had interacted with frequently or recently. I thought this would make the recommendations more personalized. However, after implementing a rough version, I realized that it could create too much user bias and might not capture general frequent trends across all users, which is important for pattern mining. I decided that sticking to traditional and proven algorithms like Apriori and FP-Growth would be a better choice for this task, so I dropped this custom approach.

In Version 3, I implemented the Apriori algorithm on the one-hot encoded transactions. I tested different support thresholds starting high with 0.5 and gradually decreasing to 0.001. I noticed that higher thresholds like 0.5 and 0.2 gave no output, while 0.01 and especially 0.005 yielded a reasonable number of patterns (around 97). That's when I realized 0.005 was a practical threshold. I also calculated precision at each step and got about 0.1368 at 0.005, which was quite consistent.

Then **In Version 4,** I shifted to FP-Growth to compare it with Apriori. I repeated the same support threshold tests. The results were similar in terms of the number of patterns and precision, but I saw that FP-Growth used much less memory and ran faster. This made me think it could be a better option for real-time systems or bigger datasets.

In Version 5, I wanted to make the model smarter by factoring in recency. I computed recency scores by measuring how recently an item was bought more recent purchases had higher scores. I integrated this into Apriori and FP-Growth, resulting in Apriori + Recency and FP-Growth + Recency models. However, I hit an issue when generating association rules. The association_rules() function raised a ValueError saying: "The input DataFrame df containing the frequent itemsets is empty." This happened because, at higher thresholds, only single-item patterns were found. I fixed this by ensuring I filtered for patterns with length greater than one.

Finally, **In Version 6,** I benchmarked all four models (Apriori, FP-Growth, Apriori + Recency, and FP-Growth + Recency) by measuring runtime, memory usage, and precision at a support threshold of 0.005. All four had the same precision of 0.1368, but FP-Growth + Recency had the lowest memory usage (16.88 MB) and a decent runtime (1.88 seconds). I chose this model because it was efficient and practical while still being accurate. After selecting FP-Growth + Recency as my final model, I moved on to Version 7, where I packaged the model logic into a single reusable function. This made it easy to integrate into the collaborative filtering system we were building as a team. I passed on this function to be connected with the main recommendation pipeline, ensuring that our final system could use frequent itemset mining along with recency awareness in a modular and scalable way.

A.2 Task 2

I contributed approximately 33.3% to the final recommender system code and report. This portion includes experimental parts, code versions of collaborative filtering recommender system, and discussions that influenced the final system. In the report, this includes an EDA (exploratory data analysis) section and collaborative filtering section. **In Version 1,** basic EDA (exploratory data analysis) was performed to understand underlying data distribution. Before analysis, the data from the training set was split into training and dev sets. Where the train set was used in EDA and training different kinds of recommender systems, and the dev set was used to evaluate the performance. Item purchase frequencies were plotted to understand how often users made purchases. Most users had fewer than five transactions, with only a few having more than 20.

In Version 2, to take this analysis a step further, raw purchase counts were plotted, which revealed that the majority of users have a minimum of two transactions. Next, user purchase overlap was analyzed, which was done by extracting items that were purchased more than twice and then filtering users who have bought those items. This showed that all the users bought an item that was bought by another user. Next, an analysis to determine sparsity was performed. The matrix had a sparsity of 96.86%, meaning that the purchases are not spread out to all products and the user purchase overlap is minimal. Hence, item-based CF was chosen as a starting point.

In Version 3, two scenarios were tested: one was not bought / not bought (the user-item matrix is binary: 1 for bought, 0 for not bought) scenario and used Jaccard similarity and a user-item matrix containing the frequency of purchase using cosine similarity. This was explored to factor in user preference for items based on the frequency. This model that included purchase frequency performed well.

In Version 4, to test for recency, like in the previous version, two models were tested: a model with recency only and a model with recency and purchase frequency factored in. In the recency model, the scores for items had higher weight, and for the second model, more recent and frequently purchased items received higher scores. The weighted scores were then log-normalized to reduce skewness and prepare the data for similarity computations. However, both the models had decreased performance. Similarly, the user-item matrix was also tested with Jaccard similarity. It also gave poorer performance.

In Version 5, the second part of Task 2 was explored by integrating frequent itemsets. Using the Apriori algorithm from the workshop, rules were generated to augment CF recommendations. Initially, rules with matching antecedents were added to CF output, excluding items already in the user's history. This approach led to significant overlap with CF recommendations. Even partial rule matching produced similar results.

Hence **In Version 6,** the strategy was readjusted by prioritizing items that were recommended by both Apriori and CF unique CF recommendations and removing previously purchased items. Then items that were already present in the user's history were then removed. Metrics like NDCG and Hit Rate were added, but results between augmented and CF-only models remained similar.

In version 7, global frequent itemsets from Task 1 were integrated. The combined approach, which prioritized common items between CF and the global set, followed by unseen global items, outperformed previous models. To explain this, purchase diversity per user was calculated, revealing that most users consistently bought only 2–3 unique items. This behavior supports why global frequency signals were effective.

Throughout the project, the ideas were critically evaluated using external sources. The initial baseline model was derived from reading external sources like [2]. While the recency-based model showed promise, however, when evaluated on the dev set, it showed poorer performance. The best model is

chosen based on its performance in the development set (appendix A).

Another major challenge I faced was integrating frequent itemset rules effectively. The data analysis revealed that purchases were heavily concentrated around a few popular items, indicating that user interactions were not evenly distributed across all products. Although all users had technically interacted with products in Version 2, further analysis during Version 7 confirmed that most users repeatedly engaged with a limited subset of items rather than exploring the full range of products. This user behavior made it challenging to leverage frequent itemsets meaningfully. As a result, the approach of directly augmenting Collaborative Filtering (CF) recommendations with rules generated from Apriori or FP-Growth was found to be less feasible (version 6), since both models came up with same predictions. Instead, the strategy was reoriented toward reordering recommendations: if an item was both frequently purchased across users and also recommended by CF for a specific user, it was prioritized higher. This adjustment was based on the fact that these items are more likely to align with user preference. Thereby improving the quality.

A.3 Task 3

I contributed approximately 33.3% to the final product and report. This part includes coding versions for integration, organizing and delivering tasks to teammates. For the report, my responsibility was to write the Executive Summary, Introduction, Related Work, Task 3: System integration section and Conclusion.

As a group leader, I did not directly deep dive into the work, I organized the first meeting for all of 3 people in the group instead. Subsequently, our group decided to adopt a Scrum-based workflow as every member of the project team is both aware of their responsibilities and how their work maps to the overall project. Due to my background as a software engineer, I realized that the Scrum framework supports the demands of iterative development (submitted versions), clear role distribution, and cohesive integration of the report and code, ensuring our team delivers a high quality final product. Speaking of which, from the beginning of this assignment, each week our team had 2 meetings (sprint reviews) to clarify each other's questions, and to plan improvements for the next sprint. In addition, each version of our code corresponds to a sprint in our Scrum workflow.

In Version 1, to kick off the integration code, I did several initial setups including:

- I made a quick draft placeholder code for loading outputs from Task 1 and Task 2.
- I left these as placeholders and had a conversation with my teammates this issue in our first Scrum meeting, proposing that the other two people would be responsible for each of these outputs.
- I drafted the generate recommendations module in advance with my user-defined pseudo-code for it.

This ensured that my initial structure was in standby mode, preparing for outputs from my teammates.

In Version 2, Initially, I loaded the train set, and I also performed basic preprocessing, such as dropping missing values and ensuring the User_id column was of integer type. However, this was just my trial to get to understand more about the dataset and the workflow. In the mean time, I loaded the sample frequent pattern mining output, `Collaborative_item_filtering_frequency.csv` from Task 1 and checked for missing values to prepare for the integration of Task 1 and Task 2 outputs. That time, in the code, I mistakenly named it as df_task_1() but it was just a draft version so it was not a big deal because I would correct it in a future version. Subsequently, I was using KNN to find

similar items, and I used Cosine similarity to measure how close two items are. Finally, I made an outline of a hybrid approach in the markdown notes to ensure that I would follow a clear vision in future work.

In Version 3, I started to make the whole Jupyter Notebook file look more organized by adding headings, subheadings, and subtitles. This action allowed me to separate Task 1 and Task 2 in a hierarchical manner. A key realization during this sprint was the need to modularize our code for better maintainability and clarity. I created a dedicated "Helper Function" block to group all functions.This ensured Separation of Concerns, making the code cleaner and easier to debug, as recommended by best practices in software engineering [15, 16]. At the same time, I focused on enhancing frequent pattern mining (Task 1) by implementing the mine frequent itemsets module to generate global frequent itemsets with a minimum support of 0.01. I also utilized the user-defined metric from Task 1 called UPPS metric (User Personalized Pattern Score) to rank frequent itemsets for a specific user. A significant challenge in this sprint was the error in task_2_output(). The error (KeyError: 'itemDescription') occurred because the frequent itemsets DataFrame lacked expected columns for pivoting. After debugging, I recognized the reason of this error right away, since my Task 1 teammate did not return the frequent itemset with the itemDescription, while in the code of my Task 2 teammate, she set the index for the itemDescription attribute.

In Version 4, Following the error from **Version 3**, I organized a meeting (sprint review) for our team to clarify each output. We decided to fix the error from Task 2 by using the correct `train_df` input, enabling collaborative filtering recommendations. In this version, for my own work, I also developed a hybrid module, combining frequent itemsets (Task 1) and collaborative filtering (Task 2). I also developed an interactive interface using ipywidgets for dynamic user input, lowered the `min_support` value to 0.005 (as this value had already been carefully justified by my Task 1 teammate).

In Version 5, during this sprint, actually, I realized that I have to cite the AI Usage that has already been described in the assignment description. I addressed the AI prompts from April 10, 2025 that I employed it for **Version 4**. Previously, the user interface prompting the entry of User_ID and option (with/without) was conducted in default window operating system on my Macbook, it was inconvenient because it had a lot of bugs (e.g., the program did not terminate when I pressed the Esc key). Therefore, I made the ipywidgets-based interface more user-friendly, clarified the "exit" input to fix termination issues, and verified the hybrid system's "with" and "without" frequent itemset recommendations (e.g., "yogurt" and "rolls/buns" for `user_id=4418` 'without' option), all while maintaining the modular structure and 0.005 support threshold.

In Version 6, In this version, we decided to remove the UPPS score after our team's sprint meeting, which determined that developing a user-defined score was too risky. After that, I integrated the recency-weighted frequent itemsets, implemented evaluation metrics (precision: 0.204/0.2014, recall: 0.1911/0.1886 for 'with'/'without'). Finally, we were ready to complete the rest of our technical report.